

Lab 04 — The Dockerfile: building your own images

Theory — the recipe, the build context, the cache, the two most expensive mistakes, and what changes when the build engine has no daemon.

Objectives

- Understand what the **build context** is and why it limits what you can copy.
- Know the essential instructions and what each one produces.
- Tell `CMD` from `ENTRYPOINT`, and *shell* form from *exec* form.
- Tell `ARG` from `ENV`.
- Order a Dockerfile to make the most of the **build cache**.

1. The build context

```
podman build -t my-api:1.0 .
```

The trailing `.` is not decoration. It names the **build context**: the directory the build is allowed to read. Two hard rules follow:

- **You can only copy files that live inside the context.** `COPY ../secrets/key.pem .` always fails — the file sits outside the boundary, and there is no workaround.
- **The entire directory belongs to the context**, including `.git/`, `node_modules/`, `target/`, log files and local configuration. A `COPY . .` drags all of it into the image.

PODMAN

Docker's client **packs** the context into an archive and **ships** it to the daemon; that is the `transferring context: 900MB` line that makes Angular builds crawl. Podman hands the build to **Buildah**, which runs inside the same process and reads the directory directly: no archive, no transfer. The context still bounds what you can copy, and `.dockerignore` still matters — not for speed, but for what ends up **in the image**. Buildah also accepts the engine-neutral names `Containerfile` and `.containerignore`.

A `.dockerignore` file at the root of the context keeps unwanted files out:

```
.git
node_modules
target
.env
```

PITFALL

Without `.dockerignore`, a `COPY . .` bakes your local secrets and the entire `.git` history **into the final image**, for anyone who pulls it to read — a classic data leak.

The Dockerfile itself can live somewhere else; point to it with `-f docker/api.Dockerfile`.

2. The essential instructions

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine # base image – always the 1st
instruction
LABEL org.opencontainers.image.source="https://git.mycompany.be/payments/api"
WORKDIR /app # creates and enters the folder
COPY target/api.jar /app/api.jar # copies from the context into the image
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75" # variable present at run time
EXPOSE 8080 # documentation: publishes NO port
USER 1000:1000 # do not run as root
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/api.jar"]
```

Instruction	Role	File layer?
FROM	Starting point	yes (the base image's layers)
RUN	Runs a command at build time	yes
COPY / ADD	Copies from the context	yes
WORKDIR, ENV, USER, EXPOSE, LABEL	Metadata	no (0B)
CMD, ENTRYPOINT	What runs at run	no
ARG	Variable for the build only	no

Three points deserve emphasis. **Prefer COPY over ADD**: ADD silently unpacks archives and downloads URLs — two implicit behaviors. **EXPOSE publishes nothing**: only -p publishes a port (lab 07). **RUN executes at build time**: RUN java -jar api.jar would start the application during the build.

REMEMBER

Spell out the full FROM: docker.io/library/eclipse-temurin:21-jre-alpine. A bare FROM eclipse-temurin:... resolves differently depending on how the building machine is configured (lab 02); at work it will read registry.internal/base/...

3. CMD versus ENTRYPOINT

Both define what runs when the container starts. They differ in how they treat arguments passed to podman run:

- CMD** provides a **default that can be replaced**. podman run my-image other-command discards the CMD.
- ENTRYPOINT** sets the **fixed** program. Arguments from podman run are **appended** to it.

```
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

podman run api executes java -jar /app/api.jar --spring.profiles.active=prod. podman run api --spring.profiles.active=dev executes the same program with the dev profile. This is the standard pattern: ENTRYPOINT pins the program, CMD supplies default arguments, and podman run --entrypoint sh -it my-image remains your escape hatch for debugging.

→ SPRING BOOT

Spring reads arguments placed after the JAR (`--spring.profiles.active=dev` , `--server.port=9090`) as properties that override `application.yml` . That is what makes the `ENTRYPOINT` + `CMD` pair so convenient: one image, a different argument per environment. Lab 08 will show that environment variables work even better.

4. Shell form and exec form

Every command can be written in two ways, and the choice is not cosmetic:

```
CMD java -jar /app/api.jar # SHELL form -> /bin/sh -c "java -jar ..."
CMD ["java","-jar","/app/api.jar"] # EXEC form -> java becomes PID 1
```

In *exec* form, the application **is** PID 1: it receives `SIGTERM` and shuts down cleanly. In *shell* form, a `/bin/sh` slips in between — the lab 03 problem all over again. A shell that stays PID 1 does not forward `SIGTERM` ; the application never sees it, `stop` waits ten seconds, then kills everything.

The key word is **stays**. What actually happens depends on which shell the image ships:

Case	PID 1	podman stop
Exec form	your application	clean, code 143
Shell form, simple command, Alpine base (busybox)	your application (the shell steps aside)	clean, code 143
Shell form, simple command, Debian/Ubuntu base (dash)	<code>/bin/sh</code>	10 s then code 137
Shell form with a pipe, an <code>&</code> , a <code>;</code>	<code>/bin/sh</code>	10 s then code 137
Entry script that launches the app without <code>exec</code>	<code>/bin/sh</code>	10 s then code 137

→ LINUX / SHELL

`/bin/sh` is not one single program: Debian and Ubuntu use `dash` , Alpine uses busybox's `ash` . When the command is the *last* one in the script, some shells replace themselves with it (an implicit `exec`); others fork a child and wait. That is why the same Dockerfile shuts down cleanly on Alpine but not on Debian.

REMEMBER

Always use the *exec* form, with JSON double quotes. If you truly need a shell, spell it out **and** use `exec` : `ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]` .

One consequence is easy to miss: in *exec* form, `$JAVA_OPTS` , `&&` , `|` and `>` are **not** interpreted — no shell is around to do it.

5. ARG versus ENV

```
ARG VERSION=1.0 # available during the build only
ENV APP_VERSION=${VERSION} # persists in the image and in the containers
```

	ARG	ENV
Visible during the build	yes	yes
Present in the final image	no	yes
Changeable at build time	<code>--build-arg VERSION=2.0</code>	no
Changeable at run time	no	<code>podman run -e APP_VERSION=...</code>

PITFALL

"ARG disappears from the image" does **not** mean "ARG is safe for secrets": the value stays visible in `podman history` and in the build cache. A password passed with `--build-arg` is a leak (lab 08).

6. The build cache

The engine processes instructions in order and caches each result. Before running an instruction it asks: have I already run this one, on top of the same parent layer? If so, it reuses the cached result (`--> Using cache`). If not, it runs the instruction — **and invalidates everything after it**. Invalidation is triggered by a change in the instruction's text, in the **content** of copied files (`COPY / ADD`), or in any earlier instruction — the effect cascades. Hence the golden rule: order your Dockerfile **from most stable to most volatile**.

```
# BAD: the code changes at every commit, so everything is rebuilt
COPY . /app
RUN mvn dependency:go-offline

# GOOD: dependencies are only re-downloaded if pom.xml changes
COPY pom.xml /app/
RUN mvn dependency:go-offline
COPY src /app/src
```

The same reasoning applies to Angular: `COPY package*.json`, then `npm ci`, and only then `COPY .`. The payoff is measured in minutes per CI build — and in deployment time too, because an unchanged layer is not re-uploaded on `push` (lab 02). To force a full rebuild, pass `--no-cache`.

7. Writing a correct RUN

```
# BAD: three layers, a 40 MB apt cache shipped in the image
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOOD: a single layer, effective clean-up
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

Clean-up must happen **in the same RUN** as the installation: deleting files in a later layer hides them without reclaiming the space (lab 02). And never leave `apt-get update` alone in its own

layer: cached for weeks, it would keep serving stale package indexes — the *cache busting* problem.

8. In the workplace

A Spring Boot back end's Dockerfile looks like this (the simple version; the multi-stage version arrives in lab 05):

```
FROM registry.internal/base/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api-*.jar app.jar
EXPOSE 8080
USER 1000:1000
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Note the choices: a **JRE** image rather than a JDK, pulled from the internal registry; a non-root `USER`; `exec` form; and a JAR copied from `target/` — so Maven ran **before** the build, in CI, the very limitation multi-stage builds will remove. On the Angular side, you never containerize `ng serve`; you containerize the output of `ng build`, served by nginx. Docker builds the same recipe in CI, Podman on your workstation: a Dockerfile is a standard.

Remember

- The `.` in `build .` names the **context**: it decides what you can copy and what a `COPY . .` drags in. `.dockerignore` is mandatory — even with Podman, where nothing is transferred.
- `EXPOSE` documents; `-p` publishes. `RUN` executes at build time, `CMD` / `ENTRYPOINT` at run time; `ENTRYPOINT` pins the program, `CMD` supplies replaceable arguments.
- *Exec* form `["prog", "arg"]`: the application is PID 1 and receives `SIGTERM`. *Shell* form: the behavior depends on the base image's shell — so avoid it. `ARG` lives only during the build, `ENV` persists in the image — neither is fit for a secret.
- Order instructions from most stable to most volatile; invalidating one instruction invalidates all that follow.
- Install and clean up in the **same** `RUN`, or the files stay in the image.

Vocabulary

build context: the directory the build may read. — `.dockerignore` / `.containerignore`: files excluded from the context. — **Containerfile**: Podman's engine-neutral name for the Dockerfile. — **Buildah**: Podman's build engine. — **exec / shell form**: the two ways to write `CMD` / `ENTRYPOINT`. — **cache busting**: deliberately invalidating the cache. — **base image**: the image named by `FROM`.